

Practical Number: 6

1. Title: Use string functions and regex for email validation, data extraction, and text analysis.

2. Software/Tools Required: Visual Studio Code (VS Code), JavaScript.

3. Theory: This JavaScript snippet demonstrates core string manipulation and regular expression techniques used for text processing.

Email Validation: Using anchored RegEx patterns (^ ... \$) with test(), we verified whether a standalone string follows standard email formatting constraints before accepting user input. Data Extraction: By leveraging pattern matching with global flags (/g) and .match(), we automatically scanned unstructured blocks of text to locate and extract embedded email addresses.

Text Analysis: Combining String methods (.toLowerCase(), .split(), .length) allowed us to analyze key structural details of text, including total character length, word counts, case transformations, and specific keyword occurrences.

4. Output:

The screenshot shows the Programiz JavaScript Online Compiler interface. The URL in the browser is <https://www.programiz.com/javascript/online-compiler/PRANATITYAGI>. The main header features a dark blue banner with the text "Stop copy pasting code you don't actually understand" and "Build the coding confidence you need to become a developer companies will fight for". There are also advertisements for Adobe Creative Cloud Pro and a "Become" button.

The code editor on the left shows a file named `main.js` with the following JavaScript code:

```
1 // 1. Input Email
2 const inputEmail = " alex.developer_2026@//domain.com ";
3
4 // Clean up input by removing accidental leading/trailing whitespace
5 const cleanEmail = inputEmail.trim();
6 console.log('Analyzing Email: "${cleanEmail}"\n');
7
8 // 2. Regular Expression for Email Validation
9 // Rules: Alphanumeric/dots/underscores before @, valid domain, 2-6 letter TLD
10 const emailRegex = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,6}$/;
11
12 // Test validation
13 const isValid = emailRegex.test(cleanEmail);
14 console.log('1. Validation Result: ${isValid ? "✅ VALID" : "❌ INVALID"}');
15
16 if (isValid) {
17     // 3. Data Extraction using String Functions
18     const atIndex = cleanEmail.indexOf("@");
19     const username = cleanEmail.substring(0, atIndex);
20     const fullDomain = cleanEmail.substring(atIndex + 1);
21
22     console.log('2. Data Extraction:');
23     console.log('   - Username: ${username}');
24     console.log('   - Full Domain: ${fullDomain}');
25
26     // 4. Text Analysis Patterns (Domain breakdown)
27     const domainParts = fullDomain.split(".");
28     const topLevelDomain = domainParts[domainParts.length - 1];
29 }
```

The output panel on the right shows the following results:

```
Analyzing Email: "alex.developer_2026@//domain.com"
1. Validation Result: ❌ INVALID
=== Code Execution Successful ===
```

5. Result/Conclusion: Through this experiment, we successfully demonstrated the application of JavaScript string functions and regular expressions for performing essential text-processing tasks. In conclusion, combining regular expressions with built-in string methods provides a robust, scalable approach to data parsing, validation, and analytics in JavaScript applications.